

CS-473: System Programming for Systems-on-Chip

Practical Work 2: The Mandelbrot Set

Lisa Suzuki (SCIPER: 408471)
Eduard Vlad (SCIPER: 374438)

October 6, 2026

1 Introduction

This report documents the implementation of two precision data types:

1. `fractal_fxpt`, a signed Q8.24 fixed-point version;
2. `fractal_myflpt`, a custom 32-bit floating-point format with inlined arithmetic.

Both reach the same image as the reference, with $\leq 0.03\%$ of pixels differing, while eliminating all calls into `libgcc`'s soft-float library.

Experimental setup. Table 1 reports the configuration under which we collected the figures below. Changes between implementations only involve changes in the number formats.

Table 1: Experimental setup.

Platform	
CPU	OR1300 (OpenRISC 1000 ISA, 5-stage pipeline, no FPU)
Instruction cache	8 kiB, direct mapped, FIFO replacement
Data cache	8 kiB, 4-way set associative, LRU, write-back
Toolchain	<code>or1k-elf-gcc 15.2.0</code>
Compiler flags	<code>-Os -DNDEBUG -D__OR1300__</code> (project default)
Workload	
Image size	512×512 pixels, <code>rgb565</code> frame buffer
Iteration limit	$n_{\max} = 64$
Viewport	$c_x \in [-2, 1)$, $c_y \in [-1.5, 1.5]$
Coordinate step	$\Delta c = 3/512 = 0.005859375$
Total iterations	$\approx 3.87 \times 10^6$ (Section 4)
Origin of the reported figures	
Static code metrics	<code>or1k-elf-size</code> and <code>or1k-elf-objdump</code> on the ELF
Run-time metrics	OR1300 profiling counters (Section ??)
Numerical accuracy	host-side replay against IEEE-754 <code>double</code> (Section 4)

2 Fixed-point implementation

2.1 Format definition

Choosing m and f requires a bound on the largest intermediate value, which we derive here for this the Mandelbrot set algorithm and viewport with $c_x \in [-2, 1)$ and $c_y \in [-1.5, 1.5]$. We compute the bounds by bounding every iteration and the trivial step at initialization.

Iteration Bounds: Let x_n, y_n be the values at the start of iteration n . The loop body computes $xx = x_n^2$, $yy = y_n^2$, updates x and y by the Mandelbrot recurrence, and only *then* tests $xx + yy < 4$. Iteration $n + 1$ is reached only if $x_n^2 + y_n^2 < 4$. Under that hypothesis, and using the maximum values $|c_x| \leq 2$, $|c_y| \leq 1.5$ for our viewport:

$$|x_{n+1}| = |x_n^2 - y_n^2 + c_x| \leq (x_n^2 + y_n^2) + |c_x| < 4 + 2 = 6, \quad (1)$$

$$|y_{n+1}| = |2x_n y_n + c_y| \leq (x_n^2 + y_n^2) + |c_y| < 4 + 1.5 = 5.5, \quad (2)$$

where the second line uses $|2xy| \leq x^2 + y^2$. The largest value ever written to `xx + yy` is bounded by

$$x_{n+1}^2 + y_{n+1}^2 < 6^2 + 5.5^2 = 66.25. \quad (3)$$

First Iteration: The first iteration where $x_1 = c_x$ and $y_1 = c_y$, yields at most $4 + 2.25 = 6.25$ and is not critical. The format must therefore represent magnitudes up to 66.25, so it needs $\lceil \log_2 66.25 \rceil = 7$ integer bits plus one sign bit. Consequently, the integer digit can be represented by $m = 8$ bits leaving 24 fractional bits:

$$\text{fxpt} = Q8.24, \quad \text{range } [-128, 128), \quad \text{resolution } 2^{-24} \approx 5.96 \times 10^{-8}.$$

We show empirically, that the value range is not exceeded: the largest value observed for `xx + yy` is 34.925 without overflow events in 3.87×10^6 iterations.

We wrap literal definitions in a C-macro (`FX()`) which is constant-folded by the compiler at compile-time. The declaration keeps the scale factor configurable and is used for all fractional literals in the program:

```
1 #define FXPT_FRAC 24 // In Q(32-FXPT_FRAC).FXPT_FRAC
2 #define FX(x) ((fxpt)((x) * (1 << FXPT_FRAC))) // compile-time float -> fxpt conversion
3 typedef int32_t (fxpt);
```

2.2 Arithmetic operations

Addition and subtraction use the native 32-bit integer addition, as both operands have the same scale. However, multiplication required additional consideration, since the resulting Q16.48 product of two Q8.24 fixed-point decimals does not fit in 32 bits:

```
1 static inline fxpt mul(fxpt a, fxpt b) {
2     return (fxpt)(((int64_t) a * b) >> FXPT_FRAC); // Q8.24 * Q8.24 = Q16.48 -> >>24 -> Q8.24
3 }
```

We leverage the fact that the 8 upper bits are always sign extension. As the maximum absolute value is bounded by 66.25, the upper 8 bits are not required to represent the resulting integer digit of the multiplication.

Shifting 24 bits rightward truncates toward $-\infty$ (as opposed to C's integer truncation) and introduces a rounding error of at most 2^{24} . In Section 4, we show that this rounding error is harmless.

2.3 Handling Over- and Underflows

We handle over- and underflows by *omitting them by construction*. This makes detecting them at runtime obsolete, as we can prove their absence.

- **Overflow** is excluded by the bound (3) together with the choice of Q8.24, as shown above. The same argument covers the intermediates: $|xx|, |yy| < 36$ and $|2xy| < 66$, all within $[-128, 128)$.
- **Underflow** represents a loss of precision, and does not wrap: a product smaller than 2^{-24} truncates to zero by shifting 24 bits rightwards.

The above guarantee is viewport-specific. A postulated zoom feature would shrink Δc but leave the bound (3) unchanged (it depends only on the escape threshold and on $|c| \lesssim 2$), so Q8.24 would remain safe against overflow. However, we later show that resolution is insufficient under this data format (cf. Section 6).

3 Custom floating-point format: myflpt

3.1 Format definition

We adopt a similar design to the suggested layout: a 32-bit word holding a sign bit, a 23-bit magnitude and an 8-bit exponent in excess-250.

bit 31	bits 30–8	bits 7–0
sign	magnitude/mantissa m (23 bits, leading one <i>explicit</i>)	exponent e (excess-250)

A non-zero value is interpreted as

$$v = (-1)^s \cdot \frac{m}{2^{23}} \cdot 2^{e-250}, \quad \frac{1}{2} \leq \frac{m}{2^{23}} < 1, \quad (4)$$

so the mantissa is normalised to the interval $[0, 1)$. The all-zero word `0x00000000` is the unique encoding of zero. Representable magnitudes span 2^{-250} up to just below $2^7 = 128$.

This choice is more suited for representing the Mandelbrot set than IEEE 754 for the following reasons:

- **Exponent in the least significant bits, mantissa above it.** This ordering makes two operations cheap. *First*, an unsigned comparison of two same-sign words is already a correct magnitude comparison, so `order()` and `is_smaller()` reduce to integer compares. *Second*, doubling a value is a single increment of the exponent field.
- **Excess-250 instead of excess-127.** The algorithm only ever handles magnitudes below 66.25, and squaring drives values toward zero. Biasing the exponent so that the representable range is $[2^{-250}, 2^5)$ in line with the maximum proven bounds for the algorithm.
Magnitudes saturate at $2^5 = 32$. Over the full image, `mul()` saturates 24 times, when it squares a value that has already escaped. This is harmless: the escape test compares against 4.0, and a saturated result of $32 > 4$ still fails the test, so the iteration terminates exactly as it should.
- **Saturation and flush-to-zero instead of $\pm\infty$ and NaN.** There is no useful reaction to an infinity in this algorithm, and supporting the special encodings would add a test to every operation. Exponent overflow therefore returns the largest representable magnitude (128, encoded `0x7FFFFFFF`) and exponent underflow returns zero. Section 3.3 shows why saturating at 128 cannot corrupt the escape test.

3.2 Arithmetic operations

All operators are implemented in `include/myflpt.h` and are declared `static inline` so that they can be inlined into the hot loop. Section 5.1 shows that this requires an explicit compiler flag to force inlining.

`float_to_myflpt()` converts an IEEE-754 `float` by re-biasing the exponent, since $1.m \cdot 2^{E-127} = 0.1m \cdot 2^{(E+124)-250}$, and by shifting the 24-bit significand `1.m` right by one to place the leading one at bit 22 of the magnitude field. It is used only on compile-time constants and is fully folded away.

`add()` and `sub()` align, operate and normalise. `order()` first swaps the operands so that $|a| \geq |b|$, which guarantees that the exponent difference $d = e_a - e_b$ is non-negative and that the subtraction of magnitudes cannot go negative. The smaller magnitude is then shifted right by d and the two are added (equal signs) or subtracted (opposite signs), after which the result is re-normalised. Both mantissas are widened by `MYFLPT_G = 8` guard bits before alignment, so that bits shifted out during alignment still contribute to the result; the guard bits are dropped at the very end. The guard $d > 23 + \text{MYFLPT_G}$ both short-circuits the case where b is negligible and avoids an undefined shift by more than 31. Subtraction is implemented as $a - b = a + (-b)$ by flipping the sign bit, which costs one XOR.

`mul()` adds the exponents ($e = e_a + e_b - 250$, the extra -250 undoing the double bias), XORs the signs, and forms the 46-bit product of the two 23-bit magnitudes in a `uint64_t`. Since both magnitudes lie in $[0.5, 1)$, the product lies in $[0.25, 1)$ and needs at most a single normalisation step, selected by one comparison against 2^{45} .

`is_smaller()` performs the escape test. Operands with different signs are ordered by sign alone; same-sign operands are ordered by the unsigned comparison of their (exponent, magnitude) fields, with the order reversed for two negative values.

Computing $2xy$. This computation deserves a remark: we first attempted to add 1 to the exponent, as it occupies the lower bits of the word. However, it is unsafe: when `mul()` saturates, the exponent field is already `0xFF` and the increment carries into the magnitude and sign fields, producing `0x80000000`. We therefore use `two_xy = add(xy, xy)` instead, which saturates correctly. We verified that both forms produce a bit-identical image at this zoom level (0 pixels differ), because the saturating case only ever occurs on the iteration that escapes; the safe form was kept because the property does not survive a change of viewport.

3.3 Limitations

We do not implement subnormals, infinities or NaNs. `float_to_myflpt()` maps a subnormal input to zero. This is deliberate and cannot be reached from constant inputs.

4 Verification

We replay both fixed and floating point libraries on our host against the IEEE-754 `double` reference (`report/verify.c`), using the exact same source headers that are compiled for the OR1300. We checked three properties: the accuracy of the individual operators, the ordering predicate, and the iteration count of every pixel of the final image.

Table 2: Worst-case relative error of the `myflpt` operators over 10^6 random operand pairs drawn from $[-4, 4]$.

Operation	worst relative error	$\approx$
<code>float_to_myflpt()</code>	1.192×10^{-7}	2^{-23}
<code>add()</code>	2.360×10^{-7}	2^{-22}
<code>sub()</code>	2.353×10^{-7}	2^{-22}
<code>mul()</code>	2.380×10^{-7}	2^{-22}
IEEE-754 <code>float</code> (for reference)	1.192×10^{-7}	2^{-23}

The bound of 2^{-22} for the arithmetic operators is exactly what the format predicts: the result magnitude can be as small as 0.5, so one truncated unit of precision 2^{-23} represents a relative error of up to 2^{-22} . `is_smaller()` was checked exhaustively over all 81 ordered pairs drawn from $\{-8, -4, -1.5, -0.25, 0, 0.25, 1.5, 4, 8\}$ and agrees with the `double` ordering on all of them.

Table 3: Agreement of the full 512×512 image with a `double` reference evaluated at the same (decoded) coordinates. “Delta” is the difference in the returned iteration count.

Version	pixels differing	share	max. delta
<code>fxpt</code> (Q8.24)	39/262 144	0.0149 %	7
<code>myflpt</code>	66/262 144	0.0252 %	19

Both versions are therefore visually indistinguishable from the reference. The differing pixels all lie on the fractal boundary, where the iteration is chaotic and an error of one unit of precision in the input changes the escape iteration. The total work is essentially identical in all three versions (3 873 537 iterations for `fxpt`, 3 873 492 for `myflpt`, 3 873 534 for the `double` reference, a spread of 0.002 %).

5 Performance

The four programs (`float`, `fxpt`, `myflpt`, `myflpt_always_inline`) were instrumented with the OR1300 profiling module (`support/include/perf.h`).

Methodology. Each configuration was measured three times, with a board reset between runs to start with cold instruction and data caches. We report cycle counts rather than the `hh:mm:ss.mmm`.

The executed-instruction counter is deterministic for a given binary. The run-time, stall and bus-idle counters do vary slightly, because two bus masters are asynchronous to the program: the SDRAM controller issues an auto-refresh every $\sim 78 \mu\text{s}$ irrespective of CPU activity.

Both loads are *common-mode*, identical in all four binaries, so they shift every row of Table 4 equally and do not distort the comparison.

Table 4: Measured on the Gecko5. Median of three runs; raw UART logs in `report/measurement/`. The derived clock is $5\,971\,036\,224/100.524 = 59.40$ MHz and is identical for all four binaries.

Version	run time [s]	run time [cycles]	executed instructions	instr. per iteration	CPI	speed-up
<code>fractal_flpt</code>	100.524	5 971 036 224	3 292 407 183	850.0	1.814	1.00×
<code>fractal_fxpt</code>	8.853	525 909 811	479 408 457	123.8	1.097	11.35 ×
<code>fractal_myflpt</code>	39.391	2 339 826 624	2 039 215 915	526.5	1.147	2.55 ×
<code>.._always_inline</code>	30.738	1 825 845 296	1 632 802 044	421.5	1.118	3.27 ×

Table 5: Memory-system counters, median of three runs. Bus occupancy is $1 - (\text{bus idle}/\text{run time})$.

Version	I\$ misses	I\$ miss penalty [cycles]	% of run time	D\$ misses	bus occupancy
fractal_flpt	24 277 779	1 647 278 253	27.6 %	16 400	40.1 %
fractal_fxpt	32	1 586	< 0.001 %	16 389	28.2 %
fractal_myflpt	52	3 506	< 0.001 %	16 389	27.9 %
.._always_inline	112	7 685	< 0.001 %	16 390	27.9 %

5.1 Always Inline

Declaring custom operators `static inline` turns out *not* to be sufficient at the project’s default optimisation level: at `-Os` as gcc keeps out-of-line copies of `add()` and `mul()` and calls them.

We therefore built a `fractal_myflpt_always_inline`, which differs from `fractal_myflpt` by placing `__attribute__((always_inline))` to function definitions.

Table 6: Effect of forcing inlining of the `myflpt` operators (measured, median of three runs).

Build	.text [B]	calls / iter.	instr. per iteration	run time [s]	I\$ misses
static inline only (-Os)	18 236	8	526.5	39.391	52
+ always_inline	20 216	3	421.5	30.738	112
<i>change</i>	+10.9 %	−5	−104.9	−22.0 %	+60

Forcing inlining removes all eight library calls, leaving only the three `__muldi3` calls that the 64-bit mantissa product requires, the same number the fixed-point version pays. The program runs $1.28\times$ faster.

6 Discussion

Using Fixed or Floating Point. For this viewport the fixed-point version wins clearly: $11.35\times$ against $3.27\times$ for the best custom-float build, a smaller binary, and marginally better accuracy (Table 3), because its 2^{-24} absolute resolution is finer than `myflpt`’s 2^{-22} relative resolution for values of order one, which is all this viewport contains.

However, with a zoom feature Δc shrinks while the escape bound stays at 4.0; Q8.24 keeps its 24 fractional bits regardless and becomes visibly quantised after 2^{20} magnification, whereas `myflpt` redistributes precision automatically.

7 Conclusion

On the board, the fixed-point version runs **11.35** $\times$ faster than the library `float` and the custom floating-point version **2.55** $\times$ faster, rising to **3.27** $\times$ once the operators are always inlined.